



COMP 412
FALL 2012

Top-Down Parsing Wrap Up
Bottom-up Parsing Intro

Copyright 2012, Keith D. Cooper & Linda Torczon, all rights reserved.

Students enrolled in Comp 412 at Rice University have explicit permission to make copies of these materials for their personal use.

Faculty from other educational institutions may use these materials for nonprofit educational purposes, provided this copyright notice is preserved.

Building Top-down Parsers for LL(1) Grammars



Given an $LL(1)$ grammar, and its FIRST & FOLLOW sets ...

- Emit a routine for each non-terminal
 - Nest of if-then-else statements to check alternate rhs's
 - Each returns true on success and throws an error on false
 - Simple, working (*perhaps ugly*) code
- Can automatically construct a recursive-descent parser

Improving matters

- Nest of if-then-else statements may be slow
 - Good case statement implementation would be better
- What about a table to encode the options?
 - Interpret the table with a skeleton, as we did in scanning

I don't know of a system
that does this ...

Building Top-down Parsers



Strategy

- Encode knowledge in a table
- Use a standard "skeleton" parser to interpret the table

Example

- The non-terminal *Factor* has 3 expansions
 - (*Expr*) or Identifier or Number
- Table might look like:

0	Goal	→	Expr
1	Expr	→	Term Expr'
2	Expr'	→	+ Term Expr'
3			- Term Expr'
4			ε
5	Term	→	Factor Term'
6	Term'	→	* Factor Term'
7			/ Factor Term'
8			ε
9	Factor	→	<u>number</u>
10			<u>id</u>
11			(Expr)

	Terminal Symbols								
	+	-	*	/	Id.	Num.	(	)	EOF
Non-terminal Symbols {	<u>Factor</u>	—	—	—	—	10	9	11	—

Cannot expand *Factor* into an operator ⇒ error

Expand *Factor* by rule 9 with input "number"

Building Top-down Parsers



Building the complete table

- Need a row for every NT & a column for every T

LL(1) Expression Parsing Table



	+	-	*	/	Id	Num	(	)	EOF
Goal	—	—	—	—	0	0	0	—	—
Expr	—	—	—	—	1	1	1	—	—
Expr'	2	3	—	—	—	—	—	4	4
Term	—	—	—	—	5	5	5	—	—
Term'	8	8	6	7	—	—	—	8	8
Factor	—	—	—	—	10	9	11	—	—

Row we built
earlier

Table differs from EaC2e because the
order of productions in *Factor* differs.

Building Top-down Parsers



Building the complete table

- Need a row for every NT & a column for every T
- Need an interpreter for the table (*skeleton parser*)



LL(1) Skeleton Parser

```
word ← NextWord()           // Initial conditions, including
push EOF onto Stack          // a stack to track local goals
push the start symbol,  $S$ , onto Stack
TOS ← top of Stack

loop forever
  if TOS = EOF and word = EOF then
    break & report success // exit on success
  else if TOS is a terminal then
    if TOS matches word then
      pop Stack              // recognized TOS
      word ← NextWord()
    else report error looking for TOS // error exit
  else                       // TOS is a non-terminal
    if TABLE[TOS,word] is  $A \rightarrow B_1 B_2 \dots B_k$  then
      pop Stack              // get rid of  $A$ 
      push  $B_k, B_{k-1}, \dots, B_1$  // in that order
    else break & report error expanding TOS
  TOS ← top of Stack
```



Building Top-down Parsers

Building the complete table

- Need a row for every NT & a column for every T
- Need a table-driven interpreter for the table
- Need an algorithm to build the table

Filling in $TABLE[X,y]$, $X \in NT$, $y \in T$

1. entry is the rule $X \rightarrow \beta$, if $y \in FIRST^+(X \rightarrow \beta)$
2. entry is *error* if rule 1 does not define

If any entry has more than one rule, G is not $LL(1)$

We call this algorithm the $LL(1)$ table construction algorithm

Recap of Top-down Parsing



- Top-down parsers build syntax tree from root to leaves
- Left-recursion causes non-termination in top-down parsers
 - Transformation to eliminate left recursion
 - Transformation to eliminate common prefixes in right recursion
- FIRST, FIRST⁺, & FOLLOW sets + LL(1) condition
 - LL(1) uses left-to-right scan of the input, leftmost derivation of the sentence, and 1 word lookahead
 - LL(1) condition means grammar works for predictive parsing
- Given an LL(1) grammar, we can
 - Build a recursive descent parser
 - Build a table-driven LL(1) parser
- LL(1) parser doesn't build the parse tree
 - Keeps lower fringe of partially complete tree on the stack



Parsing Techniques

Top-down parsers (LL(1), recursive descent)

- Start at the root of the parse tree and grow toward leaves
- Pick a production & try to match the input
- Bad "pick" $\Rightarrow$ may need to backtrack
- Some grammars are backtrack-free *(predictive parsing)*

Bottom-up parsers (LR(1), operator precedence)

- Start at the leaves and grow toward root
- As input is consumed, encode possibilities in an internal state
- Start in a state valid for legal first tokens
- Bottom-up parsers handle a large class of grammars

Bottom-up Parsing

(definitions)



The point of parsing is to construct a derivation

A derivation consists of a series of rewrite steps

$$S \Rightarrow \gamma_0 \Rightarrow \gamma_1 \Rightarrow \gamma_2 \Rightarrow \dots \Rightarrow \gamma_{n-1} \Rightarrow \gamma_n \Rightarrow \text{sentence}$$

- Each γ_i is a sentential form
 - If γ contains only terminal symbols, γ is a **sentence** in $L(G)$
 - If γ contains 1 or more non-terminals, γ is a **sentential form**
- To get γ_i from γ_{i-1} , expand some NT $A \in \gamma_{i-1}$ by using $A \rightarrow \beta$
 - Replace the occurrence of $A \in \gamma_{i-1}$ with β to get γ_i
 - In a leftmost derivation, it would be the first NT $A \in \gamma_{i-1}$

A **left-sentential form** occurs in a leftmost derivation

A **right-sentential form** occurs in a rightmost derivation

Bottom-up parsers build a rightmost derivation in reverse

Bottom-up Parsing

(definitions)



A bottom-up parser builds a derivation by working from the input sentence back toward the start symbol S

$$S \Rightarrow \gamma_0 \Rightarrow \gamma_1 \Rightarrow \gamma_2 \Rightarrow \dots \Rightarrow \gamma_{n-1} \Rightarrow \gamma_n \Rightarrow \text{sentence}$$

← bottom-up

To reduce γ_i to γ_{i-1} match some *rhs* β against γ_i then replace β with its corresponding *lhs*, A . *(assuming the production $A \rightarrow \beta$)*

In terms of the parse tree, it works from leaves to root

- Nodes with no parent in a partial tree form its *upper fringe*
- Since each replacement of β with A shrinks the upper fringe, we call it a *reduction*.
- "Rightmost derivation in reverse" processes words *left to right*

The parse tree need not be built, it can be simulated

$$|\text{parse tree nodes}| = |\text{terminal symbols}| + |\text{reductions}|$$



Finding Reductions

Consider the grammar

0	Goal	→	<u>a</u> A B <u>e</u>
1	A	→	A <u>b</u> <u>c</u>
2			<u>b</u>
3	B	→	<u>d</u>

And the input string abcde

derivation

Sentential Form	Next Reduction	
	Prod'n	Pos'n
<u>abcde</u>	2	2
<u>a</u> A <u>bcde</u>	1	4
<u>a</u> A <u>de</u>	3	3
<u>a</u> A B <u>e</u>	0	4
Goal	—	—

*The trick is scanning the input and finding the next reduction.
The mechanism for doing this must be efficient.*

*The reductions are obvious from the derivation. Of course,
building the derivation is not a practical way to find them.*



Finding Reductions

Consider the grammar

0	Goal	→	<u>a</u> A B <u>e</u>
1	A	→	A <u>b</u> <u>c</u>
2			<u>b</u>
3	B	→	<u>d</u>

And the input string abcde

parse

Sentential Form	Next Reduction	
	Prod'n	Pos'n
<u>abcde</u>	2	2
<u>a</u> A <u>bcde</u>	1	4
<u>a</u> A <u>de</u>	3	3
<u>a</u> A B <u>e</u>	0	4
Goal	—	—

*The trick is scanning the input and finding the next reduction
The mechanism for doing this must be efficient*

"Position" specifies where the right end of β occurs in the current sentential form.

While the process of finding the next reduction appears to be almost oracular, it can be automated in an efficient way for a large class of grammars.

Finding Reductions

(Handles)



The parser must find a substring β of the tree's frontier that *matches some production $A \rightarrow \beta$ that occurs as one step in the rightmost derivation* ($\Rightarrow \beta \rightarrow A$ is in RRD)

Informally, we call this substring β a *handle*

Formally,

A *handle* of a right-sentential form γ is a pair $\langle A \rightarrow \beta, k \rangle$ where $A \rightarrow \beta \in P$ and k is the position in γ of β 's rightmost symbol.

If $\langle A \rightarrow \beta, k \rangle$ is a handle, then replacing β at k with A produces the right sentential form from which γ is derived in the rightmost derivation.

Because γ is a right-sentential form, the substring to the right of a handle contains *only terminal symbols*

$\Rightarrow$ the parser doesn't need to scan (*much*) past the handle



Using Handles: a Bottom-up Parser

As with the top-down parser, we will introduce a stack to hold the upper fringe of the partially completed parse tree.

A simple *shift-reduce parser*:

```
push INVALID
token ← next_token( )
repeat until (top of stack = Goal and token = EOF)
  if the top of the stack is a handle  $A \rightarrow \beta$ 
    then // reduce  $\beta$  to  $A$ 
      pop  $|\beta|$  symbols off the stack
      push  $A$  onto the stack
  else if (token ≠ EOF)
    then // shift
      push token
      token ← next_token( )
  else // need to shift, but out of input
    report an error
```

What happens on an error?

- It fails to find a handle
- Thus, it keeps shifting
- Eventually, it consumes all input

This parser reads all input before reporting an error, not a desirable property.

Error localization is an issue in the handle-finding process that affects the practicality of shift-reduce parsers...

We will fix this issue later.

Example



0	<i>Goal</i>	$\rightarrow$	<i>Expr</i>
1	<i>Expr</i>	$\rightarrow$	<i>Expr</i> + <i>Term</i>
2			<i>Expr</i> - <i>Term</i>
3			<i>Term</i>
4	<i>Term</i>	$\rightarrow$	<i>Term</i> * <i>Factor</i>
5			<i>Term</i> / <i>Factor</i>
6			<i>Factor</i>
7	<i>Factor</i>	$\rightarrow$	<u>number</u>
8			<u>id</u>
9			(<i>Expr</i>)

A simple left-recursive form of the classic expression grammar

Bottom-up parsers work with either left-recursive or right-recursive grammars.

The obvious left-recursive grammar is left associative. The obvious right-recursive grammar is right associative.[†]

I prefer the obvious left-recursive grammar and will use it throughout the examples.

[†]We can, of course, create a left-associative, right-recursive grammar using the left-recursion removal algorithm.



Example

derivation

0	Goal	→	Expr
1	Expr	→	Expr + Term
2			Expr - Term
3			Term
4	Term	→	Term * Factor
5			Term / Factor
6			Factor
7	Factor	→	<u>number</u>
8			<u>id</u>
9			(Expr)

Prod'n	Sentential Form
—	Goal
0	Expr
2	Expr - Term
4	Expr - Term * Factor
8	Expr - Term * <id,y>
6	Expr - Factor * <id,y>
7	Expr - <num,2> * <id,y>
3	Term - <num,2> * <id,y>
6	Factor - <num,2> * <id,y>
8	<id,x> - <num,2> * <id,y>

A simple left-recursive form of the classic expression grammar

Rightmost derivation of $x = 2 * y$

Example



0	Goal	→	Expr
1	Expr	→	Expr + Term
2			Expr - Term
3			Term
4	Term	→	Term * Factor
5			Term / Factor
6			Factor
7	Factor	→	<u>number</u>
8			<u>id</u>
9			(Expr)

Prod'n	Sentential Form	Handle
—	Goal	—
0	Expr	0,1
2	Expr - Term	2,3
4	Expr - Term * Factor	4,5
8	Expr - Term * <id,y>	8,5
6	Expr - Factor * <id,y>	6,3
7	Expr - <num,2> * <id,y>	7,3
3	Term - <num,2> * <id,y>	3,1
6	Factor - <num,2> * <id,y>	6,1
8	<id,x> - <num,2> * <id,y>	8,1



parse

A simple left-recursive form of the classic expression grammar

Handles for rightmost derivation of $x = 2 * y$

Back to x - 2 * y



Stack	Input	Handle	Action
\$	<u>id</u> - <u>num</u> * <u>id</u>	none	shift
\$ <u>id</u>	- <u>num</u> * <u>id</u>		

0	Goal	→	Expr
1	Expr	→	Expr + Term
2			Expr - Term
3			Term
4	Term	→	Term * Factor
5			Term / Factor
6			Factor
7	Factor	→	<u>number</u>
8			<u>id</u>
9			(Expr)

1. Shift until the top of the stack is the right end of a handle
2. Find the left end of the handle and reduce

Back to $\underline{x} - \underline{2} * \underline{y}$



Stack	Input	Handle	Action
\$	$\underline{id} - \underline{num} * \underline{id}$	<i>none</i>	<i>shift</i>
\$ $\underline{id}$	$- \underline{num} * \underline{id}$	8,1	<i>reduce 8</i>
\$ <i>Factor</i>	$- \underline{num} * \underline{id}$	6,1	<i>reduce 6</i>
\$ <i>Term</i>	$- \underline{num} * \underline{id}$	3,1	<i>reduce 4</i>
\$ <i>Expr</i>	$- \underline{num} * \underline{id}$		

0	<i>Goal</i>	→	<i>Expr</i>
1	<i>Expr</i>	→	<i>Expr + Term</i>
2			<i>Expr - Term</i>
3			<i>Term</i>
4	<i>Term</i>	→	<i>Term * Factor</i>
5			<i>Term / Factor</i>
6			<i>Factor</i>
7	<i>Factor</i>	→	<u><i>number</i></u>
8			<u><i>id</i></u>
9			<i>(Expr)</i>

1. Shift until the top of the stack is the right end of a handle
2. Find the left end of the handle and reduce

Back to x - 2 * y



Stack	Input	Handle	Action
\$	<u>id</u> - <u>num</u> * <u>id</u>	none	shift
\$ <u>id</u>	- <u>num</u> * <u>id</u>	8,1	reduce 8
\$ <i>Factor</i>	- <u>num</u> * <u>id</u>	6,1	reduce 6
\$ <i>Term</i>	- <u>num</u> * <u>id</u>	3,1	reduce 4
\$ <i>Expr</i>	- <u>num</u> * <u>id</u>		

0	Goal	→	<i>Expr</i>
1	<i>Expr</i>	→	<i>Expr</i> + <i>Term</i>
2			<i>Expr</i> - <i>Term</i>
3			<i>Term</i>
4	<i>Term</i>	→	<i>Term</i> * <i>Factor</i>
5			<i>Term</i> / <i>Factor</i>
6			<i>Factor</i>
7	<i>Factor</i>	→	<u>number</u>
8			<u>id</u>
9			(<i>Expr</i>)

Expr is not a handle at this point because it does not occur at this point in the derivation.

While that statement sounds like oracular mysticism, we will see that the decision can be automated efficiently.

1. Shift until the top of the stack is the right end of a handle
2. Find the left end of the handle and reduce

Back to x - 2 * y



Stack	Input	Handle	Action
\$	<u>id</u> - <u>num</u> * <u>id</u>	none	shift
\$ <u>id</u>	- <u>num</u> * <u>id</u>	8,1	reduce 8
\$ <i>Factor</i>	- <u>num</u> * <u>id</u>	6,1	reduce 6
\$ <i>Term</i>	- <u>num</u> * <u>id</u>	3,1	reduce 3
\$ <i>Expr</i>	- <u>num</u> * <u>id</u>	none	shift
\$ <i>Expr</i> -	<u>num</u> * <u>id</u>	none	shift
\$ <i>Expr</i> - <u>num</u>	* <u>id</u>		

0	Goal	→	<i>Expr</i>
1	<i>Expr</i>	→	<i>Expr</i> + <i>Term</i>
2			<i>Expr</i> - <i>Term</i>
3			<i>Term</i>
4	<i>Term</i>	→	<i>Term</i> * <i>Factor</i>
5			<i>Term</i> / <i>Factor</i>
6			<i>Factor</i>
7	<i>Factor</i>	→	<u>number</u>
8			<u>id</u>
9			(<i>Expr</i>)

1. Shift until the top of the stack is the right end of a handle
2. Find the left end of the handle and reduce

Back to x - 2 * y



Stack	Input	Handle	Action
\$	<u>id</u> - <u>num</u> * <u>id</u>	none	shift
\$ <u>id</u>	- <u>num</u> * <u>id</u>	8,1	reduce 8
\$ Factor	- <u>num</u> * <u>id</u>	6,1	reduce 6
\$ Term	- <u>num</u> * <u>id</u>	3,1	reduce 3
\$ Expr	- <u>num</u> * <u>id</u>	none	shift
\$ Expr -	<u>num</u> * <u>id</u>	none	shift
\$ Expr - <u>num</u>	* <u>id</u>	7,3	reduce 7
\$ Expr - Factor	* <u>id</u>	6,3	reduce 6
\$ Expr - Term	* <u>id</u>		

0	Goal	→	Expr
1	Expr	→	Expr + Term
2			Expr - Term
3			Term
4	Term	→	Term * Factor
5			Term / Factor
6			Factor
7	Factor	→	<u>number</u>
8			<u>id</u>
9			(Expr)

1. Shift until the top of the stack is the right end of a handle
2. Find the left end of the handle and reduce

Back to $\underline{x} - \underline{2} * \underline{y}$



Stack	Input	Handle	Action
\$	$\underline{id} - \underline{num} * \underline{id}$	none	shift
\$ $\underline{id}$	$- \underline{num} * \underline{id}$	8,1	reduce 8
\$ Factor	$- \underline{num} * \underline{id}$	6,1	reduce 6
\$ Term	$- \underline{num} * \underline{id}$	3,1	reduce 3
\$ Expr	$- \underline{num} * \underline{id}$	none	shift
\$ Expr -	$\underline{num} * \underline{id}$	none	shift
\$ Expr - $\underline{num}$	$* \underline{id}$	7,3	reduce 7
\$ Expr - Factor	$* \underline{id}$	6,3	reduce 6
\$ Expr - Term	$* \underline{id}$	none	shift
\$ Expr - Term *	$\underline{id}$	none	shift
\$ Expr - Term * $\underline{id}$			

0	Goal	→	Expr
1	Expr	→	Expr + Term
2			Expr - Term
3			Term
4	Term	→	Term * Factor
5			Term / Factor
6			Factor
7	Factor	→	<u>number</u>
8			<u>id</u>
9			(Expr)

1. Shift until the top of the stack is the right end of a handle
2. Find the left end of the handle and reduce



Back to x - 2 * y

Stack	Input	Handle	Action
\$	<u>id</u> - <u>num</u> * <u>id</u>	none	shift
\$ <u>id</u>	- <u>num</u> * <u>id</u>	8,1	reduce 8
\$ Factor	- <u>num</u> * <u>id</u>	6,1	reduce 6
\$ Term	- <u>num</u> * <u>id</u>	3,1	reduce 3
\$ Expr	- <u>num</u> * <u>id</u>	none	shift
\$ Expr -	<u>num</u> * <u>id</u>	none	shift
\$ Expr - <u>num</u>	* <u>id</u>	7,3	reduce 7
\$ Expr - Factor	* <u>id</u>	6,3	reduce 6
\$ Expr - Term	* <u>id</u>	none	shift
\$ Expr - Term *	<u>id</u>	none	shift
\$ Expr - Term * <u>id</u>		8,5	reduce 8
\$ Expr - Term * Factor		4,5	reduce 4
\$ Expr - Term		2,3	reduce 2
\$ Expr		0,1	reduce 0
\$ Goal		none	accept

0	Goal	→	Expr
1	Expr	→	Expr + Term
2			Expr - Term
3			Term
4	Term	→	Term * Factor
5			Term / Factor
6			Factor
7	Factor	→	<u>number</u>
8			<u>id</u>
9			(Expr)

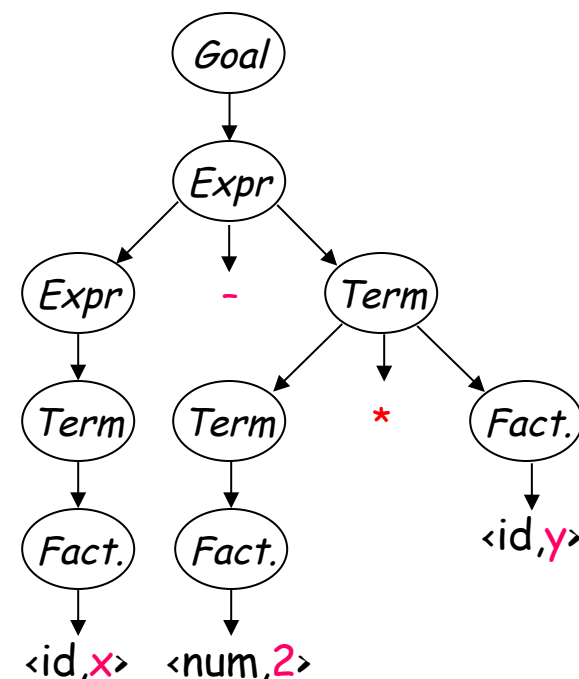
5 shifts +
9 reduces +
1 accept

1. Shift until the top of the stack is the right end of a handle
2. Find the left end of the handle and reduce



Back to $x - 2 * y$

Stack	Input	Action
\$	<u>id</u> - num * id	shift
\$ <u>id</u>	- num * id	reduce 8
\$ Factor	- num * id	reduce 6
\$ Term	- num * id	reduce 3
\$ Expr	- num * id	shift
\$ Expr -	num * id	shift
\$ Expr - num	* id	reduce 7
\$ Expr - Factor	* id	reduce 6
\$ Expr - Term	* id	shift
\$ Expr - Term *	id	shift
\$ Expr - Term * <u>id</u>		reduce 8
\$ Expr - Term * Factor		reduce 4
\$ Expr - Term		reduce 2
\$ Expr		reduce 0
\$ Goal		accept



Corresponding Parse Tree

More on Handles



Bottom-up reduce parsers find a rightmost derivation in reverse order

- Rightmost derivation $\Rightarrow$ rightmost NT expanded at each step in the derivation
- Processed in reverse $\Rightarrow$ parser proceeds left to right

These statements are somewhat counter-intuitive



More on Handles

Bottom-up parsers find a reverse rightmost derivation

- Process input left to right
 - Upper fringe of partially completed parse tree is $(NT \mid T)^* T^*$
 - The handle always appears with its right end at the junction between $(NT \mid T)^*$ and T^* (*the hot spot for LR parsing*)
 - We can keep the prefix of the upper fringe of the partially completed parse tree on a stack
 - The stack makes the position information irrelevant
- Handles appear at the top of the stack
- All the information for the decision is at the hot spot
 - The next word in the input stream
 - The rightmost NT on the fringe & its immediate left neighbors
 - In an LR parser, additional information in the form of a "state"